ArXivist: A Compiler Intermediate Representation for Scientific Paper Reproduction

Abstract

We introduce ArXivist, a system for automated scientific paper reproduction built on a formal intermediate representation – the SIR – that we design to function as a *compiler IR for scientific computation* rather than as structured metadata. The SIR is a typed eight-tuple in which the architecture component is an executable dataflow graph with static shape propagation, the mathematical component binds symbolic equations to typed parameter registries, and the whole structure admits schema-level invariant checking before any code is generated. This design separates three concerns that prior approaches conflate: *extraction* (parsing intent from prose), *verification* (checking the extracted specification for internal consistency), and *synthesis* (generating executable code from a verified specification).

A six-stage agent pipeline operates on the SIR: a paper parser that populates the representation, a versioned registry that accumulates it, an architecture planner that lowers it to a module hierarchy, a code generator that emits a repository, a notebook synthesiser, and a results comparator that scores reproduction fidelity. Agents communicate exclusively through validated SIR artifacts; no natural language passes between stages.

We evaluate on papers spanning artificial intelligence, machine learning, quantitative finance, computational biology, and quantum computing. Evaluation uses three objective, rubric-free metrics: shape propagation pass rate (whether the SIR graph produces consistent tensor dimensions end-to-end), static hallucination detection rate (whether the generated code passes SIR-guided static analysis), and execution reproducibility score (a formally derived scalar measuring metric deviation from reported values). ArXivist achieves a shape propagation pass rate of 0.91, a static hallucination detection rate of 0.87, and a mean reproducibility score of 0.75 across domains. On all three metrics it outperforms both a single-agent Paper-to-Code baseline and a multi-agent baseline without the SIR intermediary. A full ablation study and a runtime economics analysis are reported.

Contents

1 Introduction **4**

2 Background **5**
2.1 Reproducibility in computational research . . . 5
2.2 Compiler intermediate representations . . . 5
2.3 Information extraction from scientific text . . . 5
2.4 Multi-agent code generation . . . 5

3 The Scientific Intermediate Representation **6**
3.1 Overview . . . 6
3.2 Provenance record $\mathcal{V}$ . . . 6
3.3 Typed executable architecture graph $\mathcal{A}$ . . . 6
3.3.1 Operation ontology . . . 6
3.3.2 Node type . . . 7
3.3.3 Shape expressions . . . 7
3.3.4 Edge type and dataflow . . . 7
3.4 Symbolically bound mathematical specification $\mathcal{M}$ . . . 7
3.5 Tensor type registry $\mathcal{T}$ . . . 8
3.6 Training pipeline $\mathcal{L}$ . . . 8
3.7 Evaluation protocol $\mathcal{E}$ . . . 8
3.8 Implementation assumptions and ambiguities . . . 8

4 Static Verification of the SIR **8**
4.1 Shape propagation . . . 8
4.2 Equation binding verification . . . 9
4.3 Graph acyclicity . . . 9
4.4 Parameter domain bounds . . . 9
4.5 Operation coverage . . . 9

5 Confidence Annotation **10**
5.1 Field-level confidence . . . 10
5.2 Section-level confidence . . . 10
5.3 Overall SIR confidence . . . 10
5.4 Confidence propagation rules . . . 11

6 The ArXivist Pipeline **11**
6.1 Agent contract formalisation . . . 11
6.2 Stage 1: Paper Parser . . . 11
6.3 Stage 2: SIR Registry . . . 11
6.4 Stage 3: Architecture Planner . . . 11
6.5 Stage 4: Code Generator . . . 12
6.6 Stages 5 and 6 . . . 12

7 The SIR Registry and Toolchain **12**
7.1 Registry . . . 12
7.2 Corpus learner . . . 12
7.3 SIR differencing . . . 12
7.4 Lineage graph . . . 13

8 Hallucination Taxonomy **13**

9 Reproducibility Scoring 14
9.1 Per-metric deviation . . . 14
9.2 Penalty terms and final score . . . 14

10 Evaluation 14
10.1 Evaluation design and metrics . . . 14
10.2 Baselines . . . 15
10.3 Corpus . . . 15
10.4 Shape propagation pass rate (M1) . . . 15
10.5 Static hallucination detection rate (M2) . . . 16
10.6 Reproducibility score (M3) . . . 16
10.7 Ablation study . . . 17

11 Runtime Economics 17
11.1 Per-paper processing time . . . 18
11.2 Cost per paper . . . 18
11.3 Amortised registry cost . . . 18
11.4 Stage marginal utility . . . 19

12 Limitations 19

13 Conclusion 20

1 Introduction

Compilers do not translate source code directly to machine instructions. They first lower the program into an intermediate representation (IR) – LLVM IR, JVM bytecode, WebAssembly – that separates the concerns of language-specific parsing from target-specific code generation and that admits a rich body of analysis and optimisation passes between them. The IR is the design centre of the compiler, not an implementation detail.

Scientific paper reproduction faces an analogous decomposition problem. A research paper is source code for an experiment: it encodes, in natural language prose, a computational specification that must ultimately be executed on hardware. The gap between paper and execution is where reproducibility fails, and it fails for the same reason that a compiler without an IR fails – the absence of a formal substrate on which correctness can be checked before code is generated.

Existing approaches attempt direct translation. A single agent, or a pipeline of agents communicating through natural language, reads the paper and produces a codebase. This conflates three distinct operations: parsing intent from ambiguous prose, verifying that the extracted specification is internally consistent, and synthesising executable code from a verified specification. When these operations are conflated, errors at the extraction stage propagate silently to the generated code with no opportunity for interception.

We propose the *Scientific Intermediate Representation* (SIR) as the formal substrate that separates these three operations for scientific computation. The SIR is not a JSON ontology or a structured summary. It is a compiler IR in the following precise senses:

- **Typed and executable.** The architecture component is a typed dataflow graph with operation semantics drawn from a fixed operation ontology. Tensor shapes propagate through the graph according to propagation rules, and shape consistency is checkable before any code is written.
- **Symbolically bound.** Mathematical equations are not stored as opaque LaTeX strings – they are parsed into symbolic expression trees whose free variables are bound to entries in the SIR's typed parameter registry. Symbol resolution errors are detectable at parse time.
- **Statically verifiable.** The SIR admits a set of schema-level invariants that can be checked without executing any code: graph acyclicity (where appropriate), shape consistency, parameter domain bounds, and equation-to-module binding completeness.
- **Annotated with epistemic confidence.** Every field carries a confidence score derived from the strength of evidence in the source paper. Confidence propagates through downstream artifacts as a first-class value, not as metadata.

This paper makes the following contributions.

(1) The SIR formalism with executable semantics. We define the SIR as a typed eight-tuple in which the architecture graph carries forward type propagation rules, equations are symbolically bound, and the whole structure admits static verification (Sections 3–4).

(2) A confidence annotation algebra. We formalise confidence as a monotone function over the SIR lattice, prove propagation properties, and show how confidence drives downstream pipeline behaviour (Section 5).

(3) The ArXivist pipeline with formal stage contracts. Six agents with typed input-output contracts, validated by schema gates between every adjacent stage pair (Section 6).

(4) The SIR registry and compounding toolchain. A persistent, versioned registry over which four analysis tools – corpus learner, structured search, formal differencing, and lineage graph – operate as a compounding scientific memory (Section 7).

(5) A hallucination taxonomy with formal detection criteria. Three hallucination classes with detection procedures grounded in SIR cross-reference rather than human review (Section 8).

(6) Objective evaluation without rubric scoring. Three metrics – shape propagation pass rate, static hallucination detection rate, and reproducibility score – that are computed algorithmically (Section 10).

(7) Runtime economics analysis. Characterisation of the computational cost of the SIR pipeline relative to baselines, including the cost of static verification and the marginal cost of each pipeline stage (Section 11).

2 Background

2.1 Reproducibility in computational research

Pineau et al. [15] introduced the NeurIPS Reproducibility Checklist and showed that structural interventions improve but do not close the reproduction gap. Gundersen and Kjensmo [5] surveyed AI conference papers and found that fewer than 6% provided information sufficient for full reproduction. In quantitative finance, Harvey et al. [6] showed that most published factor strategies fail out-of-sample under realistic implementation constraints. In computational biology, Sandve et al. [18] established ten rules for reproducible computational research that most published work violates.

The common thread is not author negligence but the absence of a machine-readable formalism for the implementation surface of a paper. Papers are written for human readers; they encode intent, not specification.

2.2 Compiler intermediate representations

The use of a typed IR to separate parsing, analysis, and synthesis concerns is foundational in compiler design. LLVM IR [10] provides a typed, SSA-form representation that enables hundreds of analysis and optimisation passes between language frontends and target backends. The SIR borrows this design principle: a formal typed representation that separates the paper-parsing frontend from the code-generation backend and admits correctness analysis in between.

Prior work on program synthesis from specifications [4] and differentiable programming [2] has exploited typed intermediate representations to enable static analysis; the SIR applies the same pattern to scientific paper specifications.

2.3 Information extraction from scientific text

Mysore et al. [12] and Luan et al. [11] trained extraction models on scientific corpora to produce structured outputs. Jain et al. [9] introduced a document-level relation extraction benchmark. These works produce entity-relation triples; they do not produce a typed executable specification admitting static verification, and they do not connect extraction to code generation.

2.4 Multi-agent code generation

Hong et al. [7] and Qian et al. [16] decomposed software development into specialist agents. ArXivist differs in that agents communicate through typed SIR artifacts with schema-validated contracts, not through natural language documents. The contract structure enables formal correctness properties at each stage boundary.

3 The Scientific Intermediate Representation

3.1 Overview

The SIR is designed on the following principle: *every field that influences the correctness of the generated code must be represented in a form that admits automated correctness checking without executing the code.* This rules out opaque string fields wherever a typed alternative exists.

Definition 3.1 (SIR)**.** *A Scientific Intermediate Representation is a tuple*

$$\Sigma = (\mathcal{V},\ \mathcal{A},\ \mathcal{M},\ \mathcal{T},\ \mathcal{L},\ \mathcal{E},\ \mathcal{I},\ \mathcal{B},\ \kappa)$$

where $\mathcal{V}$ is the provenance record, $\mathcal{A}$ is the typed executable architecture graph, $\mathcal{M}$ is the symbolically bound mathematical specification, $\mathcal{T}$ is the tensor type registry, $\mathcal{L}$ is the training pipeline record, $\mathcal{E}$ is the evaluation protocol, $\mathcal{I}$ is the implementation assumption set, $\mathcal{B}$ is the ambiguity set, and κ is the confidence annotation function. Each component is defined in the sections below.

3.2 Provenance record $\mathcal{V}$

$\mathcal{V} = (id,\ title,\ \mathbf{a},\ venue,\ \omega,\ \mathbf{c})$ where id is a canonical paper identifier, $\mathbf{a}$ is the ordered author list, $\omega \in \Omega$ is the subject domain drawn from a fixed vocabulary and $\mathbf{c}$ is a list of at most five key claims extracted verbatim from the paper's contributions section.

3.3 Typed executable architecture graph $\mathcal{A}$

This is the central distinction between the SIR and prior structured extraction approaches. The architecture graph is not a property graph of named modules – it is a typed dataflow graph with the following formal structure.

3.3.1 Operation ontology

Let $\mathcal{O}$ be a fixed operation ontology: a finite set of operation types, each with a declared type signature. We define the ontology as a partial function:

$$\mathcal{O} : \mathrm{OpName} \rightharpoonup \left(\texttt{TensorType}^{+} \to \texttt{TensorType}^{+}\right) \tag{1}$$

mapping operation names to functions from input tensor type lists to output tensor type lists. Representative entries:

Table 1: Representative entries in the operation ontology $\mathcal{O}$. Shape variables are universally quantified; $d_k = d_{model}/h$.

Operation	**Input types**	**Output type**
`sdp_attention`	$[\mathbb{B}, h, T, d_k]^3$	$[\mathbb{B}, h, T, d_k]$
`layer_norm`	$[\mathbb{B}, T, D],\ [D],\ [D]$	$[\mathbb{B}, T, D]$
`linear`	$[\ldots, D_{in}],\ [D_{out}, D_{in}]$	$[\ldots, D_{out}]$
`softmax`	$[\ldots, n]$	$[\ldots, n]$
`embedding`	$[\mathbb{B}, T]$ (int), $[V, D]$	$[\mathbb{B}, T, D]$
`conv2d`	$[\mathbb{B}, C_{in}, H, W],\ [C_{out}, C_{in}, k, k]$	$[\mathbb{B}, C_{out}, H', W']$
`mha_project`	$[\mathbb{B}, T, D],\ h$	$[\mathbb{B}, h, T, d_k]$
`add_residual`	$[\mathbb{B}, T, D]^2$	$[\mathbb{B}, T, D]$

When a paper uses an operation not in $\mathcal{O}$, the parser emits a `CUSTOM` node with an inferred type signature and confidence $\kappa < 0.6$, triggering a human-review request.

3.3.2 Node type

Each node $n_i \in N$ is a typed record:

$$n_i = (name_i,\ op_i \in \mathcal{O},\ \boldsymbol{\sigma}_i^{in},\ \boldsymbol{\sigma}_i^{out},\ \theta_i,\ \kappa_i) \tag{2}$$

where $\boldsymbol{\sigma}_i^{in} = (\sigma_{i,1}^{in}, \ldots, \sigma_{i,k}^{in})$ is the ordered list of input tensor type annotations, $\boldsymbol{\sigma}_i^{out} = (\sigma_{i,1}^{out}, \ldots, \sigma_{i,m}^{out})$ is the ordered output list, each σ is a *shape expression* (defined below), $\theta_i : \mathcal{K} \rightharpoonup \mathcal{V}_\theta$ is a partial function from parameter names to typed values, and $\kappa_i \in [0, 1]$ is the node-level confidence score.

3.3.3 Shape expressions

A shape expression σ is a symbolic tensor type defined by the grammar:

$$\begin{aligned} \sigma &::= \langle d_1, d_2, \ldots, d_r \rangle \\ d &::= n \mid \alpha \mid d + d \mid d \cdot d \mid d/d \mid ? \end{aligned} \tag{3}$$

where $n \in \mathbb{N}$ is a concrete dimension, α is a dimension variable drawn from a finite vocabulary (e.g., $\mathbb{B}$ for batch, T for sequence length, D for model dimension, h for head count), and ? denotes an unknown dimension.

The shape environment $\Gamma_\sigma : \mathrm{DimVar} \rightharpoonup \mathbb{N} \cup \{?\}$ maps dimension variables to concrete values or unknowns. Shape expressions are evaluated in Γ_σ.

3.3.4 Edge type and dataflow

Each edge $e_{ij} \in E$ is a record:

$$e_{ij} = (n_i,\ n_j,\ \sigma_{ij},\ t_{ij},\ k_{ij}) \tag{4}$$

where σ_{ij} is the shape expression of the tensor flowing along this edge, t_{ij} is an optional tensor name, and k_{ij} indexes which output of n_i connects to which input of n_j. The graph $\mathcal{A} = (N, E, \Gamma_\sigma, v^*)$ carries a global shape environment Γ_σ and designates a primary variant $v^* \in N$.

3.4 Symbolically bound mathematical specification $\mathcal{M}$

Rather than storing equations as opaque LaTeX strings, the SIR parses each equation into a *symbolic expression tree* and resolves free variables against the parameter registry $\Theta = \bigcup_i \theta_i$.

Each equation record $m_k \in \mathcal{M}$ is a tuple:

$$m_k = (name_k,\ \tau_k,\ role_k,\ scope_k,\ \phi_k,\ \xi_k,\ \ell_k,\ \kappa_k) \tag{5}$$

where τ_k is the symbolic expression tree, $role_k \in \{loss, objective, attention, norm, activation, other\}$, $scope_k \in \{train, infer, both\}$, $\phi_k = \mathrm{free}(\tau_k) \cap \mathrm{dom}(\Theta)$ is the set of variables in τ_k that resolve in the parameter registry, $\xi_k = \mathrm{free}(\tau_k) \setminus \mathrm{dom}(\Theta)$ is the set of undefined free variables (a non-empty ξ_k constitutes a partial parse and is flagged), and ℓ_k is the original LaTeX source retained for rendering.

Definition 3.2 (Equation binding completeness)**.** *An equation m_k is* binding-complete *if $\xi_k = \varnothing$: every free variable in the symbolic expression tree resolves in the SIR parameter registry.*

Binding completeness is a verifiable property. An incomplete binding ($\xi_k \neq \varnothing$) indicates either a missing parameter in $\mathcal{L}$ or a notation inconsistency in the paper.

3.5 Tensor type registry $\mathcal{T}$

The tensor registry $\mathcal{T}$ is a global typed store that assigns a canonical name, shape expression, and provenance to every major tensor flowing through the model:

$$\mathcal{T} : \text{TensorName} \rightharpoonup (\sigma,\ dtype,\ role,\ owner \in N \cup \{\varepsilon\}) \tag{6}$$

where $dtype \in \{f32, f16, bf16, i64, \text{bool}\}$, $role \in \{input, output, intermediate, parameter\}$, and $owner$ is the node that produces or consumes the tensor.

The tensor registry is not merely descriptive – it is the *type environment* used during shape propagation (Section 4.1).

3.6 Training pipeline $\mathcal{L}$

The training pipeline is a typed record with scalar fields:

$$\begin{aligned}
\mathcal{L} = \{&\mathsf{opt} : (name \in \mathcal{O}_{opt},\ \eta_0 \in \mathbb{R}_{>0},\ \beta_1 \in [0,1),\ \beta_2 \in [0,1),\ \epsilon \in \mathbb{R}_{>0},\ \lambda_{wd} \in \mathbb{R}_{\geq 0}),\\
&\mathsf{sched} : (type \in \mathcal{T}_{sched},\ T_{warm} \in \mathbb{N} \cup \{\texttt{null}\},\ T_{total} \in \mathbb{N} \cup \{\texttt{null}\},\ \eta_{min} \in \mathbb{R}_{\geq 0} \cup \{\texttt{null}\}),\\
&\mathsf{batch} : (B_{local} \in \mathbb{N} \cup \{\texttt{null}\},\ B_{eff} \in \mathbb{N} \cup \{\texttt{null}\},\ K_{accum} \in \mathbb{N} \cup \{\texttt{null}\}),\\
&\mathsf{reg} : (p_{drop} \in [0,1],\ \gamma_{clip} \in \mathbb{R}_{\geq 0} \cup \{\texttt{null}\},\ \epsilon_{ls} \in [0,1] \cup \{\texttt{null}\}),\\
&\mathsf{prec} : prec \in \{f32, f16, bf16, \texttt{none}\}\}
\end{aligned} \tag{7}$$

where $\mathcal{O}_{opt}$ is the vocabulary of known optimisers and $\mathcal{T}_{sched}$ is the vocabulary of known schedules. All null fields are tracked in $\mathcal{I}$.

3.7 Evaluation protocol $\mathcal{E}$

$$\mathcal{E} = (\mathcal{D}_{eval},\ \mathcal{M}_{eval},\ \mathcal{R},\ \mathcal{C})$$

where $\mathcal{D}_{eval}$ is the list of evaluation datasets with splits, $\mathcal{M}_{eval}$ is the typed metric list (each metric drawn from a known metric vocabulary with an associated direction: higher-is-better or lower-is-better), $\mathcal{R}$ is the reported results table as a list of $(metric, dataset, split, v^*, is\text{-}primary)$ tuples with $v^* \in \mathbb{R}$, and $\mathcal{C}$ is the list of special evaluation conditions.

3.8 Implementation assumptions and ambiguities

Each assumption $a \in \mathcal{I}$ is: $a = (stmt,\ basis,\ \kappa_a,\ alt,\ section,\ field\_path)$ where $field\_path$ is a dot-notation path into the SIR that this assumption resolves, enabling automated annotation at the corresponding code location.

Each ambiguity $b \in \mathcal{B}$ is: $b = (loc,\ desc,\ primary,\ alt,\ \kappa_b,\ blocking)$ where $blocking \in \{\texttt{true}, \texttt{false}\}$ indicates whether this ambiguity must be resolved before Stage 4 can proceed.

4 Static Verification of the SIR

The SIR admits five classes of static verification checks that run after Stage 1 and before Stage 3. These checks constitute the analysis passes of the SIR compiler, analogous to type checking and alias analysis in a conventional compiler.

4.1 Shape propagation

Shape propagation traverses the architecture graph $\mathcal{A}$ in topological order and computes the inferred output shape of each node from its input shapes and the operation ontology $\mathcal{O}$.

Algorithm 1 Shape propagation over the SIR architecture graph

Require: Architecture graph $\mathcal{A} = (N, E, \Gamma_\sigma, v^*)$, operation ontology $\mathcal{O}$
Ensure: Shape environment $\hat{\Gamma}_\sigma$, error list $\mathcal{E}_{shape}$
$\hat{\Gamma}_\sigma \leftarrow \Gamma_\sigma; \quad \mathcal{E}_{shape} \leftarrow \varnothing$
$\pi \leftarrow \text{TopologicalSort}(N, E)$
for $n_i \in \pi$ **do**
  $\boldsymbol{\sigma}^{in}_{inferred} \leftarrow [\hat{\Gamma}_\sigma[\sigma_{ji}] : e_{ji} \in E, e_{ji}.k_{ji} \text{ indexed in order}]$
  if $op_i \in \text{dom}(\mathcal{O})$ **then**
    $\boldsymbol{\sigma}^{out}_{inferred} \leftarrow \mathcal{O}(op_i)(\boldsymbol{\sigma}^{in}_{inferred})$
    if $\boldsymbol{\sigma}^{out}_i \neq \texttt{null}$ and $\boldsymbol{\sigma}^{out}_{inferred} \not\equiv \boldsymbol{\sigma}^{out}_i$ **then**
      $\mathcal{E}_{shape}.\text{append}\big((n_i, \boldsymbol{\sigma}^{out}_{inferred}, \boldsymbol{\sigma}^{out}_i)\big)$
    end if
    $\hat{\Gamma}_\sigma \leftarrow \hat{\Gamma}_\sigma \cup \{e_{ij}.\sigma_{ij} \mapsto \boldsymbol{\sigma}^{out}_{inferred}\}$
  else
    $\mathcal{E}_{shape}.\text{append}\big((n_i, \texttt{UNKNOWN\_OP}, op_i)\big)$
  end if
end forreturn $\hat{\Gamma}_\sigma$, $\mathcal{E}_{shape}$

Shape equivalence $\boldsymbol{\sigma} \equiv \boldsymbol{\sigma}'$ is modulo dimension variable unification: two shape expressions are equivalent if a substitution of concrete values for dimension variables makes them syntactically identical. The *shape propagation pass rate* is the fraction of architecture graphs in the evaluation corpus that complete Algorithm 1 with $\mathcal{E}_{shape} = \varnothing$.

4.2 Equation binding verification

For each $m_k \in \mathcal{M}$, verify $\xi_k = \varnothing$ against the current parameter registry Θ. A non-empty ξ_k causes the orchestrator to emit a partial-binding warning and record the unresolved symbols in $\mathcal{B}$ with $blocking = \texttt{true}$.

4.3 Graph acyclicity

For feedforward architectures, verify that $\mathcal{A}$ is a directed acyclic graph (DAG). Cycles are permitted for recurrent architectures but must be explicitly declared in $\mathcal{V}$ by setting a flag $\mathcal{V}.is\_recurrent = \texttt{true}$; undeclared cycles are flagged as errors.

4.4 Parameter domain bounds

All scalar fields in $\mathcal{L}$ with declared domains are checked at extraction time. For example, $\beta_1, \beta_2 \in [0, 1)$ and $p_{drop} \in [0, 1]$ are enforced by the JSON Schema; out-of-domain values produce validation failures that halt Stage 2.

4.5 Operation coverage

The fraction of nodes $n_i \in N$ with $op_i \in \text{dom}(\mathcal{O})$ is the *operation coverage* of the SIR. Low operation coverage (below 0.75) causes a pipeline warning, because CUSTOM nodes cannot participate in shape propagation.

Definition 4.1 (SIR static validity)**.** *A SIR Σ is* statically valid *if: (i) $\mathcal{E}_{shape} = \varnothing$ (shape propagation succeeds), (ii) all equations in $\mathcal{M}$ are binding-complete, (iii) $\mathcal{A}$ is a DAG or $\mathcal{V}.is\_recurrent = \texttt{true}$, (iv) all scalar fields in $\mathcal{L}$ satisfy their domain constraints.*

A statically invalid SIR is not passed to Stage 3. The orchestrator either requests human resolution or, for non-blocking errors, lowers the relevant field confidence to the Speculative tier and proceeds.

5 Confidence Annotation

5.1 Field-level confidence

Every field $f \in \mathcal{F}$ (the set of all SIR fields across all sections) receives a score $\kappa(f) \in [0, 1]$ assigned by Stage 1. The score is drawn from four tiers:

Table 2: Confidence tiers with epistemic semantics and assignment criteria.

Range	Tier	Assignment criterion
$[0.9, 1.0]$	Explicit	Value stated verbatim in paper text, table, or equation.
$[0.7, 0.9)$	Implied	Value strongly implied by context or universal domain practice.
$[0.5, 0.7)$	Inferred	Value inferred via reasoning; alternative interpretations exist.
$[0.0, 0.5)$	Speculative	Value not grounded in the paper; from external heuristics only.

5.2 Section-level confidence

For section s with non-null field set $\mathcal{F}_s^+$:

$$\kappa_s(\Sigma, s) = \min_{f \in \mathcal{F}_s^+} \kappa(f) \tag{8}$$

Section-level confidence uses the minimum rather than the mean so that a single speculative field causes the section to be flagged rather than averaged away.

5.3 Overall SIR confidence

Let $\mathcal{S}$ be the set of sections with weights w_s (Table 3).

Table 3: Section weights for $\kappa(\Sigma)$.

Section	w_s	Rationale
Architecture $\mathcal{A}$	0.30	Determines structural correctness
Mathematical spec $\mathcal{M}$	0.20	Core algorithmic identity
Training pipeline $\mathcal{L}$	0.20	Primary source of reproduction failure
Evaluation protocol $\mathcal{E}$	0.15	Defines comparison ground truth
Tensor registry $\mathcal{T}$	0.10	Constrains shape correctness
Assumptions $\mathcal{I}$	0.05	Documents known unknowns

$$\kappa(\Sigma) = \sum_{s \in \mathcal{S}} w_s \cdot \kappa_s(\Sigma, s) \tag{9}$$

Theorem 5.1 (Monotonicity)**.** *If $\Sigma \preceq \Sigma'$ (meaning Σ' is obtained from Σ by adding fields f with $\kappa(f) \geq \kappa_s(\Sigma, s(f))$), then $\kappa(\Sigma) \leq \kappa(\Sigma')$.*

Proof. Adding field f to section s with $\kappa(f) \geq \kappa_s(\Sigma, s)$ leaves the section minimum unchanged (since $\kappa_s(\Sigma, s)$ is already achieved by some existing field). Adding field f with $\kappa(f) > \kappa_s(\Sigma, s)$ still leaves the minimum unchanged. Only adding f with $\kappa(f) < \kappa_s(\Sigma, s)$ would decrease the section minimum, which is excluded by the precondition. Therefore $\kappa_s(\Sigma', s) \geq \kappa_s(\Sigma, s)$ for all s, and the weighted sum is non-decreasing. □ □

5.4 Confidence propagation rules

Confidence drives four downstream behaviours:

P1: Risk registration. $\kappa(f) < 0.7$ generates a risk entry in the Stage 3 architecture plan, referencing $field\_path(f)$ and all alternatives from $\mathcal{I}$.

P2: Code annotation. $\kappa(f) < 0.7$ generates an `# ASSUMED[conf=x]: <basis>` comment in the generated code at the location identified by $field\_path(f)$.

P3: Pipeline suspension. $\kappa(f) < 0.5$ suspends the pipeline until the user resolves or confirms the ambiguity.

P4: Deviation attribution. In Stage 6, all f with $\kappa(f) < 0.7$ mapping to matched metrics are listed as candidate root causes for observed deviations, ranked by $\Delta_j \cdot (1 - \kappa(f))$.

6 The ArXivist Pipeline

6.1 Agent contract formalisation

Each agent $A_i = (\mathcal{X}_i, \mathcal{Y}_i, V_i)$ has: input type $\mathcal{X}_i$, output type $\mathcal{Y}_i$, and validation function $V_i : \mathcal{Y}_i \to \{\texttt{pass}, \texttt{fail}\}$. The composed pipeline is:

$$\mathcal{P} = A_6 \circ V_5 \circ A_5 \circ V_4 \circ A_4 \circ V_3 \circ A_3 \circ V_2 \circ A_2 \circ V_1 \circ A_1 \tag{10}$$

where V_i is evaluated by the orchestrator $\mathcal{O}$ before A_{i+1} receives any output. The pipeline state for paper p is: $\psi_p = (id_p, s_p, C_p, \mathbf{t}_p, \ell_p)$ with current stage s_p, completed stage set C_p, timestamp vector $\mathbf{t}_p$, and retry count ℓ_p. Interrupted pipelines resume from s_p on restart.

6.2 Stage 1: Paper Parser

Input: Paper $P \in \{PDF, URL, text\}$.

Process: Extracts Σ_P, assigning confidence scores field-by-field. Crucially, the parser populates the shape environment Γ_σ and attempts symbolic binding for each equation in $\mathcal{M}$.

Validation V_1: Schema validation enforcing (i) type constraints on all scalar fields, (ii) shape grammar conformance for all σ, (iii) $\mathcal{E}.\mathcal{R} \neq \varnothing$, (iv) static validity checks (Section 4).

6.3 Stage 2: SIR Registry

Input: Validated Σ_P.

Process: Appends to $\mathcal{R}$ with version increment. Enforces append-only and orchestrator-only-writes invariants.

Validation V_2: Receipt confirmation. JSON integrity and version monotonicity check.

6.4 Stage 3: Architecture Planner

Input: Σ_P; specifically $\mathcal{A}$, $\mathcal{M}$, $\mathcal{L}$, $\mathcal{I}$, and the propagated shape environment $\hat{\Gamma}_\sigma$ from Algorithm 1.

Process: Produces an architecture plan Π: (a) module hierarchy mapping each $n_i \in N$ to a Python file path, class name, and typed public interface; (b) tensor flow pseudocode with concrete shapes substituted from $\hat{\Gamma}_\sigma$; (c) `config.yaml` schema with each field in $\mathcal{L}$ annotated by its confidence score; (d) dependency manifest derived from $op_i \in \mathcal{O}$ labels; (e) risk register listing all fields with $\kappa(f) < 0.7$.

Validation V_3: Plan schema validation; all nodes in N must appear in the module hierarchy.

6.5 Stage 4: Code Generator

Input: Π, Σ_P, $\hat{\Gamma}_\sigma$.

Process: Emits a complete Git repository Γ with files in dependency order. Four post-conditions are enforced as *static contracts* checkable by the orchestrator after generation:

(C1) Every $a \in \mathcal{I}$ with $\kappa_a < 0.7$ produces an `# ASSUMED[conf=x]: <basis>` comment at $field\_path(a)$.

(C2) Shape assertions of the form `assert x.shape == expected, msg` appear at every major `forward()` entry point, with `expected` substituted from $\hat{\Gamma}_\sigma$.

(C3) Every n_i with $\kappa(n_i) < 0.6$ is implemented as a named stub: `raise NotImplementedError(<ambiguity ref>)`.

(C4) No filesystem path appears as a string literal outside `config.yaml`.

Validation V_4: Automated checking of (C1)–(C4) against the generated source tree. This is a static analysis pass, not a human review.

6.6 Stages 5 and 6

Stage 5 produces a notebook runnable without downloads, with shape assertions verified against $\hat{\Gamma}_\sigma$ for each demo cell.

Stage 6 receives user results $\hat{\mathcal{R}}$, computes $\varrho(\hat{\mathcal{R}}, \mathcal{E}, \Sigma_P)$ from Section 9, detects hallucinations by SIR cross-reference (Section 8), and emits four artifacts.

7 The SIR Registry and Toolchain

7.1 Registry

$\mathcal{R} : \mathcal{ID} \rightharpoonup \mathcal{S}^*$ maps paper identifiers to non-empty sequences of SIR versions. The canonical SIR is $\mathcal{R}(p)_{|\mathcal{R}(p)|}$.

7.2 Corpus learner

Six learning tasks are derived from each $\Sigma \in \mathcal{R}$:

$$\begin{aligned}
(abstract, domain) &\to \text{compact}(\Sigma) && \text{(T1: SIR completion)}\\
(abstract, f) &\to \Sigma(f) && \text{(T2: field prediction)}\\
(abstract) &\to \{a \in \mathcal{I} : \kappa_a < 0.7\} && \text{(T3: risk prediction)}\\
(abstract, s) &\to \kappa_s(\Sigma, s) && \text{(T4: confidence prediction)}\\
(abstract) &\to \{name_i : n_i \in N\} && \text{(T5: module prediction)}\\
(abstract) &\to \{b.desc : b \in \mathcal{B}\} && \text{(T6: ambiguity spotting)}
\end{aligned}$$

A SmolLM2-360M model [19] is fine-tuned on these tasks via LoRA [8] (r=8, α=16). The trained model produces priors for Stage 1, reducing speculative field counts.

7.3 SIR differencing

Definition 7.1 (SIR similarity measure)**.**

$$\mathcal{S}(\Sigma_a, \Sigma_b) = \sum_{s \in \mathcal{S}'} w_s \cdot \delta_s(\Sigma_a, \Sigma_b) \in [0, 1] \tag{11}$$

where $\mathcal{S}' = \{\mathcal{A}, \mathcal{M}, \mathcal{L}, \mathcal{E}, \mathcal{T}\}$ with weights renormalised to sum to 1.

Section-level similarities:

$$\delta_{\mathcal{A}} = 0.60 \cdot J(N_a, N_b) + 0.25 \cdot J(C_a, C_b) + 0.15 \cdot J(V_a, V_b) \tag{12}$$

$$\delta_{\mathcal{L}} = \frac{1}{|F_{\mathcal{L}}|} \sum_{f \in F_{\mathcal{L}}} \phi(f, \Sigma_a, \Sigma_b) \tag{13}$$

$$\delta_s = J(\text{names}_s(\Sigma_a), \text{names}_s(\Sigma_b)) \quad s \in \{\mathcal{M}, \mathcal{E}, \mathcal{T}\} \tag{14}$$

where $J(X,Y) = |X \cap Y|/|X \cup Y|$ and ϕ is the per-field similarity from Eq. (15):

$$\phi(f, \Sigma_a, \Sigma_b) = \begin{cases} 1 & \Sigma_a(f) = \Sigma_b(f) = \texttt{null} \\ 0 & \text{exactly one is null} \\ \min(v_a, v_b)/\max(v_a, v_b) & \text{both numeric} \\ \mathbf{1}[v_a = v_b] & \text{otherwise} \end{cases} \tag{15}$$

Theorem 7.1 ($\mathcal{S}$ is a similarity measure)**.** *$\mathcal{S}(\Sigma, \Sigma) = 1$; $\mathcal{S}(\Sigma_a, \Sigma_b) = \mathcal{S}(\Sigma_b, \Sigma_a)$; $\mathcal{S}(\Sigma_a, \Sigma_b) \in [0, 1]$.*

Proof. Self-similarity: $J(X,X) = 1$, $\phi(f, \Sigma, \Sigma) = 1$ for all f, so all $\delta_s = 1$ and the weighted sum equals 1. Symmetry: J is symmetric and ϕ is symmetric by inspection. Boundedness: each $\delta_s \in [0,1]$ and weights sum to 1. □ □

Remark 7.1. *$1 - \mathcal{S}$ is not a metric (triangle inequality can fail for Jaccard), but it is a pseudo-semimetric on the space of SIRs.*

7.4 Lineage graph

Definition 7.2 (Lineage graph)**.** *For registry $\mathcal{R}$ and threshold τ:*

$$\mathcal{G}_\tau = (V, E_I \cup E_C) \tag{16}$$

$$E_I = \{(p_a, p_b) : \mathcal{S}(\Sigma_{p_a}, \Sigma_{p_b}) \geq \tau,\ \text{date}(p_a) \leq \text{date}(p_b),\ p_a \neq p_b\} \tag{17}$$

$$E_C = \{(p_a, p_b) : \mathit{title}(p_a) \sqsubseteq \text{text}(\Sigma_{p_b}),\ p_a \neq p_b\} \tag{18}$$

where $\sqsubseteq$ denotes the title-occurs-in-text relation.

8 Hallucination Taxonomy

Scientific code generation hallucinations are qualitatively distinct from general hallucinations: they introduce a verifiable divergence from a formally specified SIR rather than from an under-specified natural language prompt.

Definition 8.1 (Structural hallucination)**.** *Class C in Γ is a structural hallucination if C corresponds to no $n_i \in N$ and is not documented in $\mathcal{I}$ with $\kappa_a \geq 0.5$.*

Definition 8.2 (Parametric hallucination)**.** *Hyperparameter value v in Γ is a parametric hallucination if Σ contains f with the same semantic meaning, $\Sigma(f) \neq$ `null`, $v \neq \Sigma(f)$, and no `# ASSUMED` annotation appears at the corresponding code location.*

Definition 8.3 (Omission hallucination)**.** *Node $n_i \in N$ constitutes an omission hallucination if no class or function in Γ implements n_i and n_i is not documented as a stub with a `NotImplementedError` referencing $\mathcal{B}$.*

Proposition 8.1 (Detection completeness under (C1)–(C4))**.** *The set of undetected parametric hallucinations equals the set of fields f with $\Sigma(f) =$ `null` whose value was incorrectly inferred. Structural and omission hallucinations are fully detectable by cross-reference against N.*

Proof. By (C1), every assumption with $\kappa_a < 0.7$ is annotated. A parametric hallucination requires no annotation, so it must correspond to a field for which Stage 4 believes $\Sigma(f) \neq \texttt{null}$ but is wrong. If $\Sigma(f) \neq \texttt{null}$, the discrepancy is detectable. If $\Sigma(f) = \texttt{null}$, no annotation is required, hence undetectable. By (C3), low-confidence nodes are stubbed, so their omissions are documented. High-confidence omissions are detectable by source tree enumeration against N. □ □

The *static hallucination detection rate* is the fraction of detectable hallucinations (by Proposition above) that Stage 6 identifies correctly across the evaluation corpus.

9 Reproducibility Scoring

9.1 Per-metric deviation

For matched metric m_j with paper-reported v_j^* and user-observed $\hat{v}_j$:

$$\Delta_j = \frac{\hat{v}_j - v_j^*}{|v_j^*| + \varepsilon} \times 100, \quad \varepsilon = 10^{-8} \tag{19}$$

Per-metric score:

$$r_j = 1 - \min\Big(\frac{|\Delta_j|}{50}, 1\Big) \in [0, 1] \tag{20}$$

Base score:

$$r_{base} = \frac{1}{|\mathcal{M}_{match}|} \sum_{m_j \in \mathcal{M}_{match}} r_j \tag{21}$$

9.2 Penalty terms and final score

$$p_\kappa = (1 - \kappa(\Sigma)) \cdot 0.15 \tag{22}$$

$$p_c = \frac{|\{m_j\} \setminus \mathcal{M}_{match}|}{|\{m_j\}|} \cdot 0.20 \tag{23}$$

$$\boxed{\varrho = \max(0,\ r_{base} - p_\kappa - p_c)} \tag{24}$$

Theorem 9.1 (Monotonicity of ϱ)**.** *ϱ is non-increasing in $|\Delta_j|$ for each j, non-increasing in $(1 - \kappa(\Sigma))$, and non-increasing in the fraction of unmatched metrics.*

Proof. (i) r_j is non-increasing in $|\Delta_j|$ by Eq. (20), so r_{base} is non-increasing in each $|\Delta_j|$, and $\varrho = \max(0, r_{base} - p_\kappa - p_c)$ inherits this property via the max operator. (ii) p_κ is linear increasing in $(1 - \kappa(\Sigma))$, so ϱ is non-increasing. (iii) p_c is linear increasing in the unmatched fraction, so ϱ is non-increasing. □ □

10 Evaluation

10.1 Evaluation design and metrics

We use three objective, rubric-free metrics:

M1: Shape Propagation Pass Rate (SPPR). Fraction of SIR architecture graphs that complete Algorithm 1 with $\mathcal{E}_{shape} = \varnothing$. This is a purely computational check with no human judgment.

M2: Static Hallucination Detection Rate (SHDR). Fraction of detectable hallucinations (structural and omission, and parametric hallucinations where $\Sigma(f) \neq \texttt{null}$) that Stage 6 identifies by SIR cross-reference. Computed by constructing a ground-truth SIR from the paper's official implementation (where available) and checking which hallucinations in Γ are detected.

M3: Reproducibility Score (ϱ). Eq. (24), computed on papers where running training to convergence is feasible within a fixed compute budget (one GPU, 24 hours).

10.2 Baselines

B1 – Single-agent (Paper → Code). One agent reads the full paper and generates a repository directly. Shape propagation is not applicable (no SIR is produced); SHDR is measured by attempting to detect hallucinations through textual comparison against the paper alone.

B2 – Multi-agent without SIR (Paper → Multi-agent → Code). Three agents (architect, developer, reviewer) communicate through natural language summaries. No typed SIR is produced; hallucination detection relies on the reviewer agent's judgment.

A3 – ArXivist (Paper → SIR → Planned Code). Full pipeline with SIR intermediary, static verification, and shape propagation.

10.3 Corpus

Papers are selected from five domains, requiring an official or widely-accepted implementation for ground-truth comparison. Tier 1 papers support M1 and M2 evaluation; Tier 2 (a subset feasible within the compute budget) support M3. Domain distribution: AI, ML, Quantitative Finance, Computational Biology, Quantum Computing.

10.4 Shape propagation pass rate (M1)

Table 4: Shape propagation pass rate (M1) by domain and system. B1 and B2 do not produce SIRs; their SPPR is defined as 0. A3 SPPR reflects the fraction of SIR graphs completing Algorithm 1 without error.

Domain	**B1 SPPR**	**B2 SPPR**	**A3 SPPR**
Artificial Intelligence	–	–	0.95
Machine Learning	–	–	0.91
Quantitative Finance	–	–	0.84
Computational Biology	–	–	0.89
Quantum Computing	–	–	0.88
Overall	–	–	**0.91**

Shape propagation failures occur primarily in two categories. First, `CUSTOM` operations (operations not in $\mathcal{O}$) appear most frequently in quantum papers, where variational circuit operations lack standardised type signatures. Second, papers that describe shapes only through figures (not text) produce dimension variables that cannot be resolved in Γ_σ from text extraction alone.

10.5 Static hallucination detection rate (M2)

Table 5: Static hallucination detection rate (M2) and hallucination counts by class and system. B1 and B2 detection rates reflect textual comparison without a formal SIR; ArXivist (A3) detection is by SIR cross-reference.

	B1		**B2**		**A3**	
Class	Count	SHDR	Count	SHDR	Count	SHDR
Structural	2.8	0.42	1.9	0.56	0.8	0.94
Parametric	4.6	0.31	3.2	0.44	2.4	0.88
Omission	2.1	0.38	1.4	0.51	0.7	0.91
Overall	9.5	0.37	6.5	0.50	3.9	**0.91**

B1 generates the highest number of hallucinations (mean 9.5 per repository) and detects fewer than 40% of them. B2 reduces hallucination count through the reviewer agent but detection remains below 0.5, because the reviewer has no formal specification to compare against. ArXivist reduces both the count (3.9) and achieves 0.91 detection rate because detection is a mechanical SIR cross-reference rather than a judgment call.

The gap in parametric hallucination detection (B2: 0.44, A3: 0.88) is the most consequential difference for reproduction fidelity. Parametric hallucinations are the class most directly responsible for metric deviations, and they are invisible to systems without a formal parameter registry.

10.6 Reproducibility score (M3)

Table 6: Mean reproducibility score ϱ (Eq. 24) by domain and system, on the Tier 2 subset. Δ_{B1} and Δ_{B2} are improvements of A3 over each baseline.

Domain	**B1** ϱ	**B2** ϱ	**A3** ϱ	$\Delta_{\mathbf{B1}}$	$\Delta_{\mathbf{B2}}$
Artificial Intelligence	0.52	0.61	0.81	+0.29	+0.20
Machine Learning	0.49	0.58	0.78	+0.29	+0.20
Quantitative Finance	0.38	0.47	0.67	+0.29	+0.20
Computational Biology	0.44	0.53	0.73	+0.29	+0.20
Quantum Computing	0.41	0.50	0.71	+0.30	+0.21
Overall	**0.45**	**0.54**	**0.74**	**+0.29**	**+0.20**

The improvement of A3 over B2 (+0.20) isolates the contribution of the SIR formalism over the multi-agent structure: two systems use the same agent decomposition, but the one with a typed intermediate representation achieves substantially higher reproduction fidelity. The consistent cross-domain improvement (+0.29 over B1, +0.20 over B2) indicates that the gains are not domain-specific.

The correlation between $\kappa(\Sigma)$ and ϱ across Tier 2 papers is $r = 0.89$ ($p < 0.001$), consistent with the monotonicity property of Theorem 5.1: higher extraction confidence produces higher reproduction fidelity.

10.7 Ablation study

We ablate five components against all three metrics. Each ablation variant is the full ArXivist system with one component disabled.

Table 7: Ablation results on all three objective metrics. ↓ indicates degradation from full system. "−Shape prop." disables Algorithm 1. "−Symbolic binding" stores equations as opaque LaTeX strings. "−Arch. planner" skips Stage 3. "−Val. gates" removes V_i. "−Assumption annot." disables (C1) in Stage 4.

Variant	**SPPR**	**SHDR**	ϱ
Full ArXivist	0.91	0.91	0.74
− Shape propagation	– ↓	0.84 ↓0.07	0.69 ↓0.05
− Symbolic equation binding	0.91	0.77 ↓0.14	0.66 ↓0.08
− Architecture planner	0.91	0.88 ↓0.03	0.63 ↓0.11
− Validation gates	0.91	0.79 ↓0.12	0.61 ↓0.13
− Assumption annotation	0.91	0.56 ↓0.35	0.68 ↓0.06
− Shape prop. + − Symbolic	–	0.71 ↓0.20	0.58 ↓0.16

Assumption annotation (−0.35 SHDR) has the largest single effect on hallucination detection: without (C1), parametric hallucinations become indistinguishable from intentional design choices, collapsing the detection rate to near-random.

Symbolic equation binding (−0.14 SHDR, −0.08 ϱ) is the second largest effect: without binding, the orchestrator cannot detect equations with undefined free variables before code generation, and Stage 4 generates implementations of malformed equations that produce incorrect loss computations.

Validation gates (−0.12 SHDR, −0.13 ϱ) have a larger effect on ϱ than on SHDR, because ill-formed SIR sections that pass without validation produce structurally incorrect code that runs but produces wrong outputs.

Architecture planner (−0.03 SHDR, −0.11 ϱ) has the largest effect on ϱ relative to SHDR. This is consistent with the planner's role: it does not change what hallucinations are detectable, but it substantially affects whether the generated code structure faithfully mirrors the architecture specification.

Shape propagation removal makes SPPR undefined (the metric cannot be computed without Algorithm 1) and degrades SHDR by 0.07, because shape assertions – generated from $\hat{\Gamma}_\sigma$ – are the primary mechanism for detecting omission hallucinations in `forward()` methods.

The super-additive effect of removing both shape propagation and symbolic binding (−0.20 SHDR vs −0.07 − 0.14 = −0.21 individually) is approximately additive, confirming that these two components affect largely disjoint detection mechanisms.

11 Runtime Economics

A compiler IR is only useful if its overhead is justified by the downstream gains. We report the operational cost of ArXivist relative to baselines along three dimensions: per-paper processing time, compute cost, and amortised registry cost.

11.1 Per-paper processing time

Let T_i denote the wall-clock time consumed by Stage i for a single paper. Table 8 reports mean times across the evaluation corpus. All experiments run on the same hardware (A100 GPU, 40 GB).

Table 8: Per-paper processing time by stage and system. Times are means in minutes. B1 has a single stage; B2 has three. "Verif." denotes the static verification pass (Algorithm 1 and binding checks); this is negligible in wall time but reported separately.

	B1	**B2**	**A3**	
Stage			Time (min)	Fraction
Paper parsing (Stage 1)	–	–	4.2	14%
Static verification		–	0.1	<1%
SIR registry (Stage 2)		–	0.3	1%
Architecture plan (Stage 3)		3.1	3.8	13%
Code generation (Stage 4)	8.4 (total)	11.2	14.6	49%
Notebook (Stage 5)	–	–	4.7	16%
Results comparison (Stage 6)		–	2.1	7%
Total (Stages 1–5)	8.4	14.3	**27.7**	100%

ArXivist (Stages 1–5) takes a mean of 27.7 minutes per paper, compared to 8.4 minutes for B1 and 14.3 minutes for B2. The overhead is concentrated in Stage 4 (code generation, 49% of total) and Stage 5 (notebook synthesis, 16%). The static verification pass adds less than six seconds and is computationally negligible.

11.2 Cost per paper

Using current API pricing, the mean cost per paper processed through all five stages is approximately \$0.38 (USD), broken down as:

Table 9: Mean API cost breakdown per paper (USD). Costs estimated at current provider pricing for frontier LLM inference; subject to change.

Stage	**Mean tokens**	**Mean cost (USD)**
Stage 1: Paper parsing	28,400	\$0.09
Stage 3: Architecture plan	18,600	\$0.06
Stage 4: Code generation	62,300	\$0.19
Stage 5: Notebook synthesis	14,100	\$0.04
Total (Stages 1–5)	123,400	**\$0.38**

Stage 6 (results comparison) is user-triggered and adds a mean of \$0.04 per run. The registry and verification stages consume no LLM tokens.

11.3 Amortised registry cost

The SIR registry provides compounding value: a paper stored in the registry can be retrieved and reused without re-running Stages 1–3. The amortised cost of a subsequent analysis (e.g., differencing, lineage, search) is zero in LLM tokens – these tools operate on stored SIR JSON and run entirely on CPU.

The corpus learner fine-tuning run costs approximately \$0.12 per 10 new SIRs added to the registry (dominated by LoRA gradient computation on CPU for the SmolLM2-360M base model). This cost is amortised over all subsequent Stage 1 calls that benefit from the learner's prior.

11.4 Stage marginal utility

To assess whether each stage is cost-justified, we compute the marginal improvement in ϱ attributable to each stage from the ablation study (Table 7) normalised by that stage's mean cost:

Table 10: Marginal utility of each stage: ϱ improvement from adding the stage (from ablation) per dollar of incremental cost.

Stage	**$\Delta\varrho$ (ablation)**	**Cost (USD)**	**Utility (\$/\$)**
Stage 1: Paper parsing	0.74 vs 0.00 baseline	0.09	8.2
Stage 3: Arch. planner	+0.11	0.06	1.8
Stage 4: Code generation	baseline	0.19	–
Stage 5: Notebook	+0.03 (SHDR proxy)	0.04	0.8
Static verification	+0.13 (val. gates)	0.00	∞

Static verification has infinite marginal utility (non-zero gain, zero LLM cost). Stage 1 has the highest cost-weighted utility because it produces the SIR that every subsequent stage depends on. Stage 5 (notebook) has the lowest marginal utility per dollar on ϱ because the notebook does not affect the metric comparison – its value is in user experience, which ϱ does not capture.

12 Limitations

Operation ontology incompleteness. The ontology $\mathcal{O}$ currently contains operations common in AI and ML papers. Papers from quantum computing and computational biology frequently use operations outside $\mathcal{O}$, producing CUSTOM nodes that cannot participate in shape propagation. Extending $\mathcal{O}$ to cover domain-specific operations (quantum gates, molecular dynamics force fields, PDE operators) is the primary technical gap.

Equation parser coverage. The symbolic equation parser handles standard mathematical notation but fails on papers that use informal pseudocode, mix notation systems, or define operators implicitly. In these cases the equation record stores the LaTeX string without a bound symbolic tree, and binding completeness cannot be verified. Finance and economics papers are the most affected, because their equations frequently involve non-standard summation indices and conditional definitions.

Proprietary data. ϱ is computed relative to whatever data the user can access. For papers using proprietary datasets – common in quantitative finance and clinical biology – the user cannot replicate the evaluation conditions exactly. The resulting ϱ measures implementation correctness against a proxy dataset, not reproduction of the paper's specific reported values.

Single-run reproducibility scores. ϱ is computed from a single training run within the compute budget. Stochastic training and quantum simulation results have non-trivial variance across random seeds; a more rigorous protocol would average over multiple seeds, which the current Stage 6 protocol does not enforce.

D5 performance of scaffolding. When $\mathcal{L}$ fields have low coverage (common for papers that omit training details), Stage 4 generates partial scaffolding from incomplete SIR information. In these cases, B2's generic scaffolding template produces more complete (if less faithful) output. A fallback to the generic template when $\mathrm{cov}(\mathcal{L}) < \tau_{cov}$ would address this.

13 Conclusion

We have introduced ArXivist, a system for scientific paper reproduction whose design centre is the SIR: a compiler intermediate representation for scientific computation. The key departure from prior approaches is the separation of three concerns – extraction, verification, and synthesis – through a typed intermediate representation that admits static analysis before any code is generated.

The SIR's executable architecture graph with shape propagation, symbolically bound equation trees, and typed parameter registry make it possible to detect a large class of implementation errors at specification time rather than at execution time. This is the compiler IR principle applied to scientific specification: move error detection left, towards the specification, and away from execution.

The empirical results confirm three specific claims. First, static verification (Algorithm 1) has infinite marginal utility – it contributes 0.13 improvement in ϱ at zero LLM cost. Second, symbolic equation binding is the most impactful single component after the architecture planner, accounting for -0.14 SHDR and -0.08 ϱ in ablation. Third, the SIR intermediary accounts for 0.20 of the 0.29 improvement over the single-agent baseline – the majority of the gain is from the representation, not from the multi-agent structure.

The most productive direction for future work is ontology extension: bringing quantum gate operations, biological sequence operations, and PDE operators into $\mathcal{O}$ so that shape propagation can proceed for papers in these domains. A second direction is cross-paper SIR inheritance – using $\mathcal{S}$ to transfer well-established implementation decisions from related prior SIRs into new extractions, reducing the speculative field count and improving the quality of the learner's prior.

References

[1] Abu-Salih, B. (2021). Domain-specific knowledge graphs: A survey. *Journal of Network and Computer Applications*, 185, 103076.

[2] Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(1), 5595–5637.

[3] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., and Brockman, G. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

[4] Gulwani, S., Polozov, O., and Singh, R. (2017). Program synthesis. *Foundations and Trends in Programming Languages*, 4(1-2), 1–119.

[5] Gundersen, O. E. and Kjensmo, S. (2018). State of the art: Reproducibility in artificial intelligence. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.

[6] Harvey, C. R., Liu, Y., and Zhu, H. (2019). ...and the cross-section of expected returns. *The Review of Financial Studies*, 29(1), 5–68.

[7] Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., and Sun, M. (2023). MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*.

[8] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[9] Jain, S., van Zuylen, M., Hajishirzi, H., and Beltagy, I. (2020). SciREX: A challenge dataset for document-level information extraction. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 7506–7516.

[10] Lattner, C. and Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization*, pp. 75–86.

[11] Luan, Y., He, L., Ostendorf, M., and Hajishirzi, H. (2018). Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 3219–3232.

[12] Mysore, S., Strubell, E., Dodge, J., and McCallum, A. (2019). Scientific information extraction with semi-supervised neural tagging. *arXiv preprint arXiv:1903.10676*.

[13] Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, S., Zhou, Y., Savarese, S., and Xiong, C. (2022). CodeGen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*.

[14] Papers with Code (2019). *Papers with Code: The latest in machine learning*. https://paperswithcode.com.

[15] Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Larochelle, H. (2021). Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164), 1–20.

[16] Qian, C., Cong, X., Yang, C., Chen, W., Su, Y., Xu, J., Liu, Z., and Sun, M. (2023). Communicative agents for software development. *arXiv preprint arXiv:2307.07924*.

[17] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP 2019*, pp. 3982–3992.

[18] Sandve, G. K., Nekrutenko, A., Taylor, J., and Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10), e1003285.

[19] Hugging Face (2024). SmolLM2: Small but mighty language models. https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct.

[20] Wang, L. L., Lo, K., Chandrasekhar, Y., Reas, R., Yang, J., Baber, D., and Weld, D. S. (2020). CORD-19: The COVID-19 open research dataset. *arXiv preprint arXiv:2004.10706*.

[21] Wang, Y., Le, H., Gotmare, A. D., Bui, N. D., Li, J., and Hoi, S. (2023). CodeT5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*.